

MERN Job Board Platform

Finalized Technical Decisions & Team Action Plan • WEB II Project (v2.1 Lock-Down)

This document contains the locked-down technical decisions, architectural rules, route ordering, status transition state-machines, and parallel team task divisions finalized for implementation. No further architectural refactoring or technology changes should occur during the coding sprint.

1. Finalized Architectural & Technical Decisions

- **Standardized API Naming:** Strictly use `GET /api/jobs/mine` (not `/employer/my-jobs`) and `GET /api/applications/me` (not `/my-applications`) for consistent, canonical REST conventions.
- **Critical Route Ordering:** In Express, `router.get('/mine', ...)` **MUST** be registered BEFORE `router.get('/:id', ...)`. Otherwise, Express will match 'mine' as an `:id` parameter and crash the query.
- **Application Status State-Machine:** Allowed transitions are strictly: `PENDING → REVIEWED`, `ACCEPTED`, `REJECTED` and `REVIEWED → ACCEPTED`, `REJECTED`. Direct jumps from `PENDING` to `ACCEPTED/REJECTED` are permitted. Any other value or reverse transition returns `400 Bad Request`.
- **Job Closing Business Rule:** When an employer closes a job (`status: 'CLOSED'`), the job remains viewable publicly for historical context and applicant tracking, but new applications are rejected with `400 Bad Request`. Existing applications remain accessible to the employer.
- **Mandatory Application Fields:** Applications require: `job (ObjectId)`, `applicant (ObjectId)`, `coverLetter` (required, min 20 chars), and `resumeLink` (required, valid HTTP/HTTPS URL). Applications with missing fields are rejected with `400 Bad Request`.
- **Middleware Pruning:** Omit `express.urlencoded()` and `express.static()`. Include strictly: `express.json()`, `cors({ origin, credentials: true })`, and `cookieParser()`.
- **Cookie & CORS Architecture:** Frontend Axios instance configured with `withCredentials: true`. Backend CORS explicitly allows frontend origin (`http://localhost:5173`) and `credentials: true`. Cookie attributes: `httpOnly: true`, `sameSite: 'lax'`, `secure: (production)`.
- **Minimal Profile Management:** Keep profile management focused strictly on: View profile, Edit name, Edit company (for employers), and Change password (verified with `bcrypt`). Do not add avatars, resume storage, or social links.
- **Focused Dashboard Statistics:** Keep dashboards functional: Seeker (Total, Pending, Reviewed, Accepted, Rejected); Employer (Total Jobs, Open Jobs, Total Applicants, Pending, Accepted, Rejected). Powered by MongoDB aggregation pipelines.
- **Standard Email Regex:** Use standard general RFC email formatting regex. Do not restrict email domains to specific providers (no forced Gmail/Yahoo).
- **Database Level Constraints:** 1) Compound unique index on Application schema: `{ job: 1, applicant: 1 }` returning `409 Conflict`. 2) MongoDB TTL index on Session schema: `{ expiresAt: 1 }, { expireAfterSeconds: 0 }`.
- **Password Security & Hashing:** `bcrypt` with fixed salt rounds = 10. Passwords stored only as `passwordHash`.
- **Updates & Verbs:** Use `PUT` consistently for all resource modifications (Job edits, Status updates, Profile updates).

2. Application Status Transition Rules

Current Status	Allowed Next Statuses	Backend Enforcement
PENDING	REVIEWED, ACCEPTED, REJECTED	Employer can review or directly accept/reject
REVIEWED	ACCEPTED, REJECTED	Final candidate evaluation decision
ACCEPTED	None (Final State)	Terminal state; changes rejected with 400 Bad Request
REJECTED	None (Final State)	Terminal state; changes rejected with 400 Bad Request

3. Parallel Team Work Breakdown & Ownership

Member / Role	Primary Responsibilities	Core Deliverables
Member 1 (Auth & Sessions)	User model, bcrypt hashing, Session model with TTL index, authentication middleware (requireAuth), role middleware (requireRole), authentication routes	server/models/User.js server/models/Session.js server/middleware/auth.js server/routes/auth.js
Member 2 (Jobs & Search)	Job model, Job CRUD endpoints with strict ownership checks, routing (routing before /:id), search query regex, filter logic	server/routes/jobs.js Ownership authorization logic
Member 3 (Applications & Review)	Application model with compound unique index ({job, applicant}), application routes, application validation (resumeLink URL regex), 409 Conflict & 400 Bad Request handlers	server/models/Application.js server/routes/applications.js 409 Conflict & 400 Bad Request handlers
Member 4 (Frontend & UI)	React application setup (Vite), React Router setup, AuthContext state management/AuthContext (withCredentials: true), ProtectedRoute	client/src/services/api.js client/src/components/ProtectedRoute.jsx Dashboard & Job pages

4. Security-Specific Testing Matrix

Category	Test Scenario	Expected Status & Behavior
Authentication	Login with invalid password	401 Unauthorized ('Invalid email or password')
Authentication	Access protected route without cookie	401 Unauthorized ('Authentication required')
Authentication	Access protected route with expired session	401 Unauthorized (Session deleted, cookie cleared)
Authentication	Logout destroys session	200 OK (Session deleted from MongoDB collection)
Authorization	Guest attempts to create job	401 Unauthorized
Authorization	Job Seeker attempts to create job	403 Forbidden ('Requires role: EMPLOYER')
Authorization	Employer A attempts to edit Employer B's job	403 Forbidden ('You do not own this job')
Authorization	Employer A views applicants for Employer B's job	403 Forbidden ('Unauthorized applicant access')
Business Rule	Job Seeker applies twice to same job	409 Conflict ('You have already applied for this job')
Business Rule	Job Seeker applies to CLOSED job	400 Bad Request ('Job posting is closed')
Business Rule	Invalid status value passed to update	400 Bad Request ('Invalid status value or transition')
Business Rule	Malformed MongoDB ObjectId parameter	404 Not Found or 400 Bad Request handled gracefully

5. Immediate Next Actions (Phase 1 Execution)

- **Action 1 (Scaffolding):** Initialize Vite React app in `client/` and Express server in `server/`.
- **Action 2 (Dependencies):** Backend installs: `express mongoose bcrypt cookie-parser cors dotenv` and `nodemon` (dev). Frontend installs: `react-router-dom axios`.
- **Action 3 (Database Connection):** Configure `server/config/db.js` and connect to MongoDB.
- **Action 4 (Seed Script):** Prepare `server/seed.js` with the 4 demo accounts (Sara, Dawit, Alazar, Beth) and 8 diverse jobs.
- **Action 5 (Core Sprints):** Member 1 launches Auth & Sessions, Member 2 builds Job CRUD, Member 3 handles Applications, Member 4 prepares React layouts.